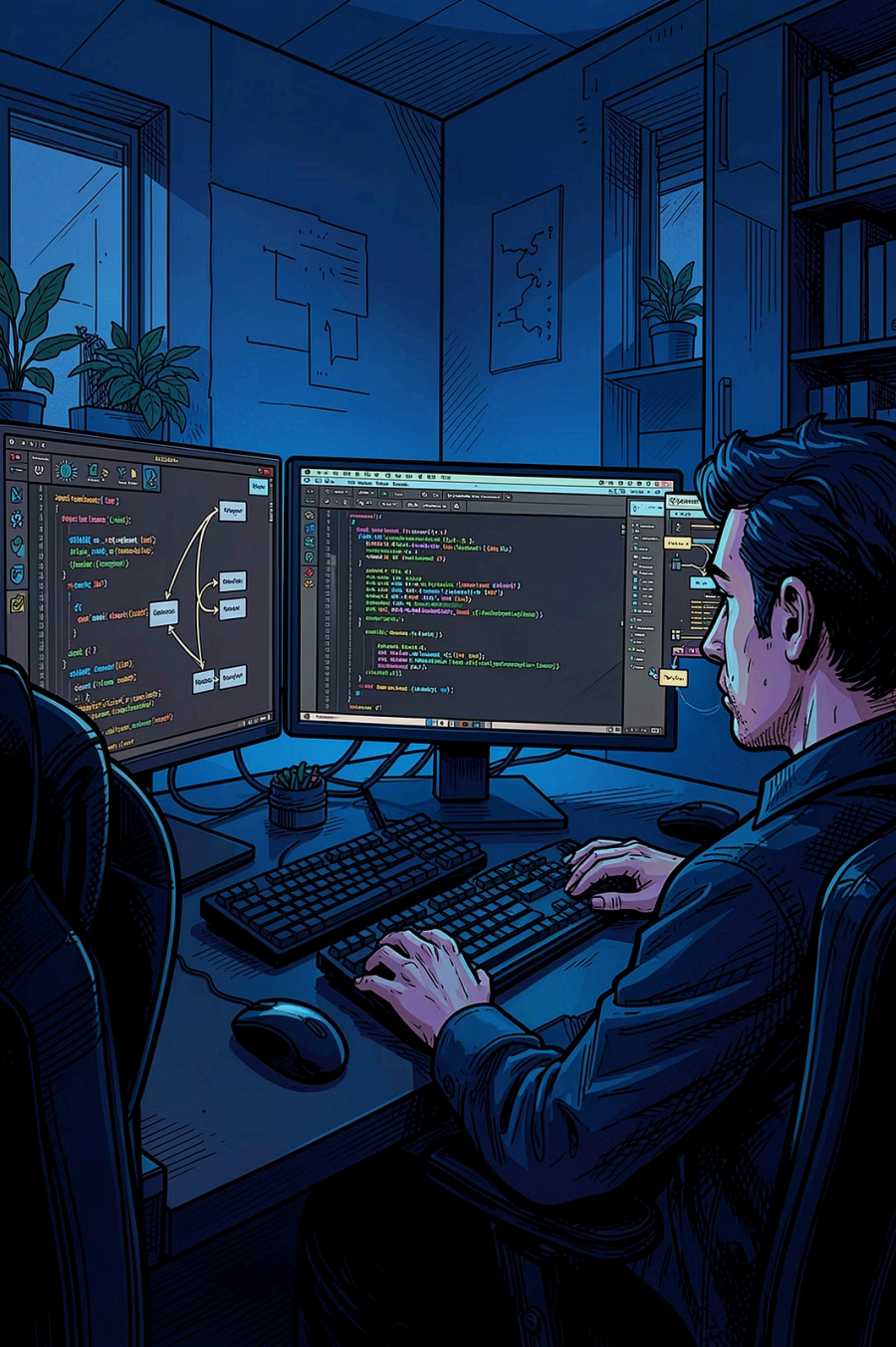




BACKEND DEVELOPMENT · INFOSEC

CI/CD & GitHub Actions Fundamentals

How modern engineering teams automate builds, tests, and deployments — and why it matters for software security.



Without automation, every step relied on human discipline – and humans make mistakes.

Forgotten test runs, skipped steps, and misconfigured environments were common.

"Works on my machine" –
different environments produced
different results.

Bugs surfaced late, sometimes only after reaching production.

🔍 **Key question:** Can we automate this entire process?

What Is CI/CD?



CI — Continuous Integration

Every code change is automatically built and tested. Problems surface within minutes, not days.

CD — Continuous Delivery / Deployment

Code that passes CI is automatically prepared for — or pushed to — production.

Goal: Automate software delivery so teams ship faster, safer, and with less manual effort.

Continuous Integration in Practice

Every code change triggers an automated verification gate. If any check fails, the change is blocked before it can cause harm.



Push Code

The diagram illustrates the Continuous Integration process flow using three horizontal arrows. The first arrow, labeled 'Push Code', is dark blue. The second arrow, labeled 'Run Checks', is a medium blue. The third arrow, labeled 'Pass or Fail', is a lighter blue. Each arrow is longer than the one above it, and they are all pointing to the right, indicating a sequential process.

Run Checks

Pass or Fail

Early detection is the core security benefit — vulnerabilities and regressions are caught before they reach production.

Delivery vs. Deployment

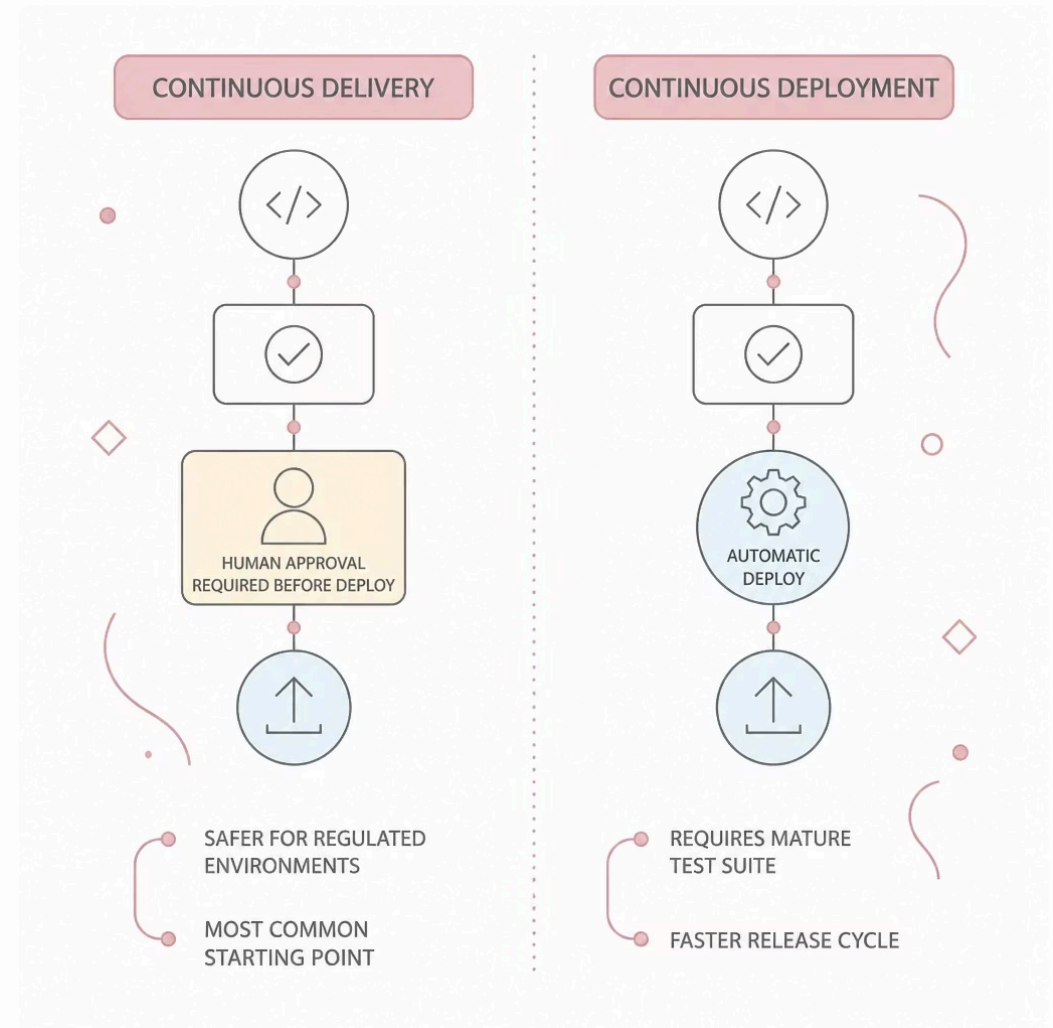
Continuous Delivery

Code is **always deployable**.

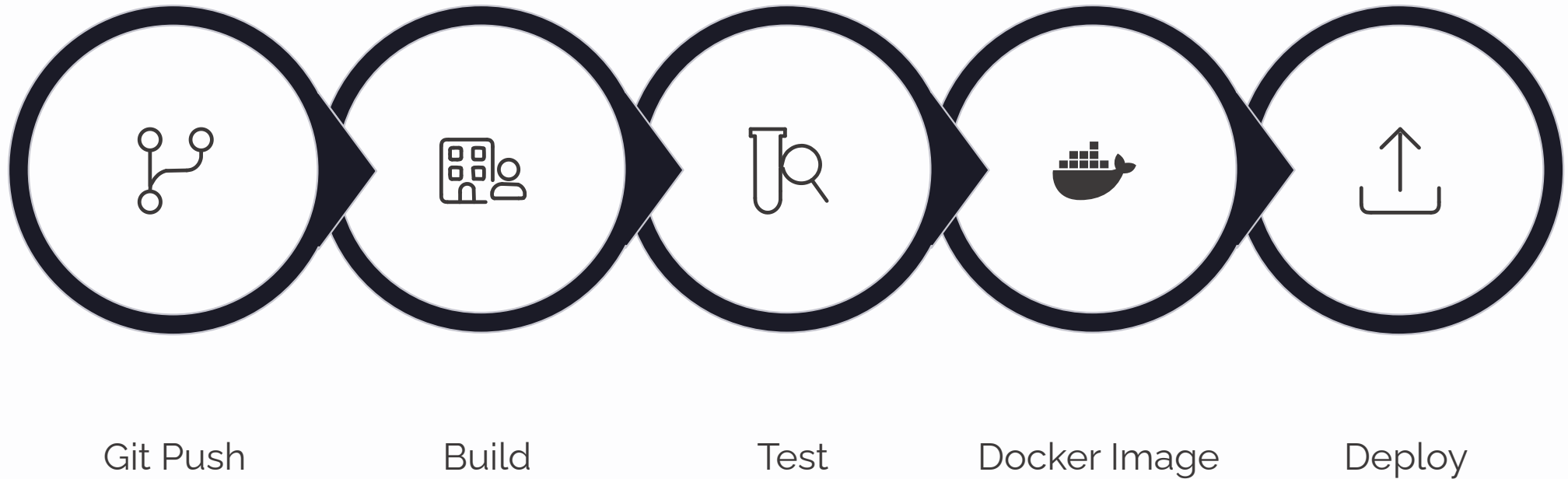
A human reviews and approves each release before it goes live. Most teams start here.

Continuous Deployment

Every passing change is **deployed automatically**. No manual gate. Requires high test coverage and mature processes.



A Typical CI/CD Pipeline



Each stage acts as a quality gate. If any step fails, the pipeline halts and the team is notified immediately — preventing broken code from progressing.

Why GitHub Actions?



Built Into GitHub

No extra infrastructure. Workflows live alongside your code in the same repository.



Full CI/CD Pipelines

Define build, test, and deploy steps in a single YAML file.



Docker & Scheduled Jobs

Build container images, run scheduled tasks, and automate recurring operations.



GitHub Actions: Workflow Anatomy

O1

Trigger

What starts the workflow – a push, pull request, or schedule.

02

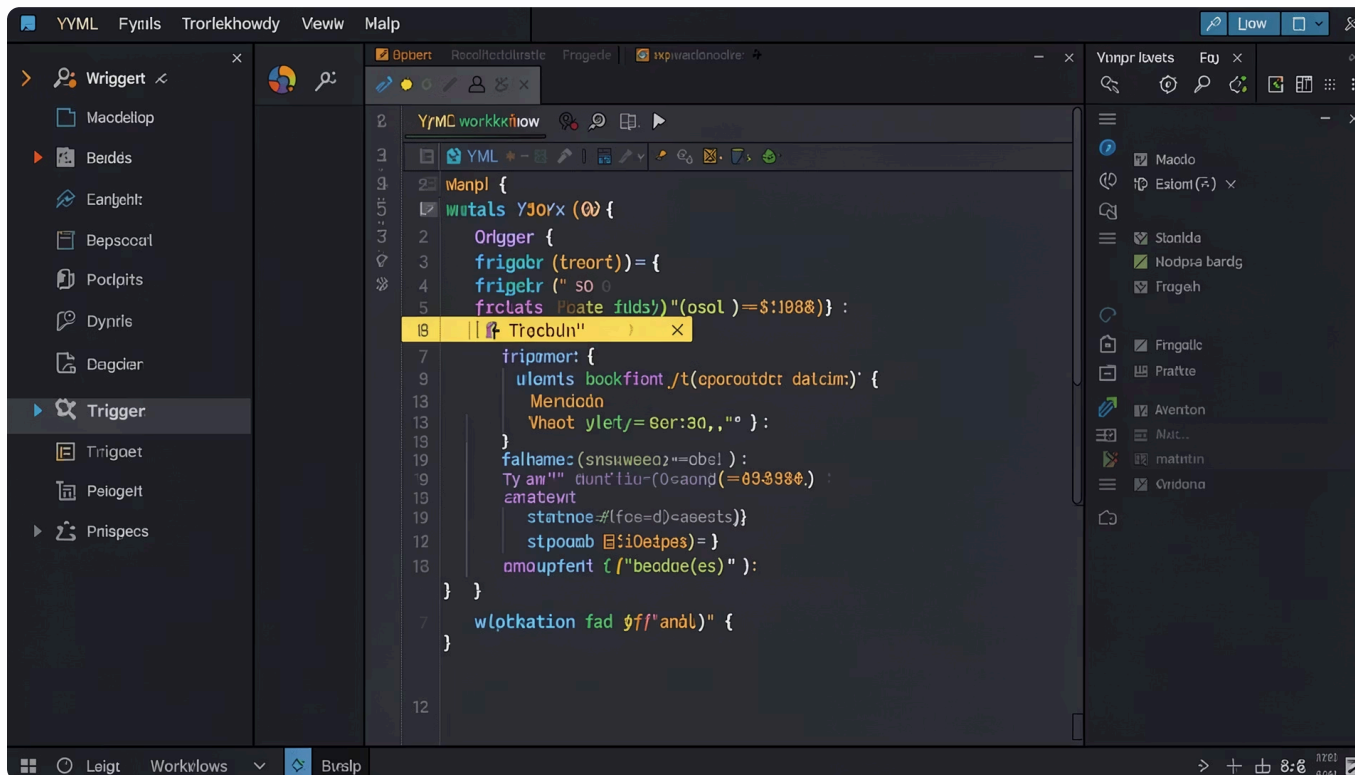
Job

Runs on a fresh VM (e.g., ubuntu-latest).
Jobs can run in parallel or sequence.

03

Steps

Individual commands or actions – checkout code, install dependencies, run tests.



 **Mini ASM Example:** Pull Request triggers Go Build → Unit Tests → Lint → Docker Build. Only healthy code merges.

Example: A Real GitHub Action

```
name: CI
on: pull_request

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
      - run: go test ./...
```

What This Does

- Triggers on every **pull request**
- Runs on a fresh **Ubuntu VM**
- Checks out your code
- Sets up Go environment
- Runs all unit tests

If tests fail, the PR **cannot be merged** — enforcing quality at the gate.